



In the Name of God
Computer-Aided Design of Digital Systems – Fall 1404
Project 1: RTL Design of a Hash Generator



Designers: Amirhossein Karili – Kimia Firouzbakhsh – Hatef Rezaei

Project Goal:

This project, through the hardware implementation of a Hash Generator at the RTL level, aims to review and reinforce the concepts of Datapath and Controller design using the Huffman method, the use of Handshaking mechanisms, and the execution of the simulation process to verify and confirm the system's functionality.

Problem Statement:

A hash is a fixed-length value generated from variable-length input data, designed so that the smallest change in the input causes a significant change in the hash value. This property allows the hash to act as a digital signature for data and makes it highly effective in detecting unintended changes or tampering with data.

Furthermore, a hash is designed so that recovering the original data from the hash value is infeasible, and the probability of two different pieces of data having the same hash is minimized.

In this project, the 128-bit input message is divided into four 32-bit words, and after executing a 64-round algorithm, a 128-bit hash output is produced.

Design and Implementation:

The design consists of four main parts:

- Pseudo-random number generation section
- Hash generation section
- Communication and coordination between the two sections
- Verification

1. Pseudo-Random Number Generation Section

This section is responsible for generating pseudo-random numbers, which are used as indices for selecting specific words from the message. The random-number generation algorithm is based on a shift register with bitwise operations such as XOR applied to specific bits. On every clock cycle, this module generates a new value based on specific bits combined via XOR; the value appears random but actually follows a completely deterministic algorithm.

You must design a module that, upon observing one complete pulse on the **rnd_start** signal, stores the 6-bit number on its input bus into an internal register and shifts it according to Figure 1. After six shift operations, it places the two most-significant bits, in the same order, onto its 2-bit output bus and sets the **rnd_done** signal to 1 for one clock cycle.

For this part, you must design the datapath and controller for this module and verify it. For example, for the input 000111, this module must produce the output 10.

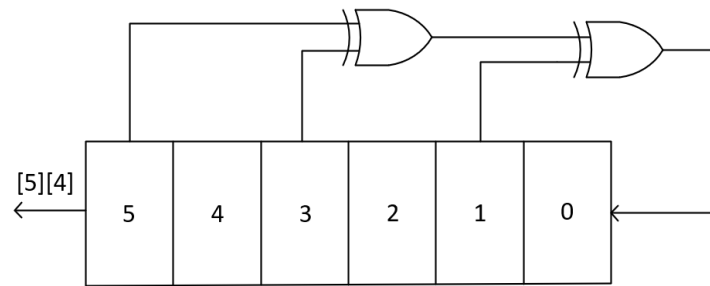


Figure 1. Internal register and shift mechanism

2. Hash Generation Section

The input message is defined as a 128-bit block consisting of four 32-bit words. At each round, the index value generated by the pseudo-random section selects a specific word from the message. This selected word is combined with the current hash value.

You must design a module that, upon observing one complete pulse on the **start** signal, stores the 128-bit message on its input bus into an internal register file and processes it over 64 rounds according to Figure 2. At the end, it places the values of the four 32-bit registers A, B, C, D – with A occupying the most-significant position – onto its 128-bit output bus, and sets the **done** signal to 1 for one clock cycle.

For this part, you must design the datapath and controller for this module and verify it. For example, for the input [41a801a8e81df62b14a661b85c97bf45], this module must produce the output [c6f6c75d2bbbb9a586cf3291347acdce].

```

Initialize:
  a0 := 0x67452301
  b0 := 0xefcdab89
  c0 := 0x98badcfe
  d0 := 0x10325476

Break 128-bit message into four 32-bit words M[0..3]

A := a0
B := b0
C := c0
D := d0

for i from 0 to 63 do
  if 0 ≤ i ≤ 15 then
    F := (B and C) or ((not B) and D)
  else if 16 ≤ i ≤ 31 then
    F := (D and B) or ((not D) and C)
  else if 32 ≤ i ≤ 47 then
    F := B xor C xor D
  else if 48 ≤ i ≤ 63 then
    F := C xor (B or (not D))

  x := i
  repeat 6 times:
    fb := (bit5 of x) xor (bit3 of x) xor (bit1 of x)
    x := (x << 1) or fb
    rnd := (bit5 of x shifted left by 1) OR (bit4 of x)

  F := F + A + constant[i] + M[rnd]

  A := D
  D := C
  C := B
  B := B + leftrotate(F, step[i])
end for

digest := A || B || C || D

```

Figure 2. Overall hash generation algorithm, including the random part

Hint: The constant values are located in the memory file **constant.mem**. You must use these values via a ROM; however, the step constant values should be defined and used directly within the rotate module:

```
// step specifies the per-round shift amounts
step[ 0..15] := { 7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22 }
step[16..31] := { 5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20 }
step[32..47] := { 4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23 }
step[48..63] := { 6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21 }
```

Figure 3. Constant values s for rounds 0 through 63

3. Communication and Coordination Between the Sections

The random section and the hash section must be connected to each other, and the hash section must command the random section – using the **rnd_start** and **rnd_done** signals – to generate a 2-bit random number for word selection, based on the main loop's iteration counter.

4. Verification

The workflow of this program for processing a message is provided in the file **example.txt**. You can use this file to debug your program.

Points to consider during implementation:

- Both sections (random and hash) must have a datapath and a controller designed in the Huffman style, and must be modular.
- The connection between these two sections must be correctly implemented in a top-level module called **Top**.
- The state machine used in the controllers must be a **Moore** machine.
- Each required operation must be expanded into an independent module.
- Having complex or duplicate modules is not allowed.
- The final Top module must return to its initial state after processing a message and be ready to receive and process the next message.

Other Notes

- This assignment is to be done in groups of two.
- Upload your files and report, named <SID>_HW1_CAD.zip, to the relevant location on the course page before the deadline.
- If you have any questions, you may contact the designers by email.
- Correct variable naming, clean code and comments, and parameterized module inputs can, to some extent, compensate for shortcomings elsewhere in the code.
- The goal of this assignment is for you to learn! If cheating is detected, it will be dealt with according to the course rules.

Good luck